

JYOTHISWAR YALLA

StockFlow Backend Case Study Solution (Java - Spring Boot)

Part 1: Code Review and Debugging

The provided API attempts to create a product and its inventory record. However, several issues were identified that could cause failures in a real-world system.

1. Input validation is missing. Required fields such as name and SKU are directly accessed without checks.
2. SKU uniqueness is not enforced, which may result in duplicate product entries.
3. Two separate database commits are used. If the second operation fails, the system will have incomplete data.
4. The design incorrectly ties a product to a single warehouse, even though products can exist in multiple warehouses.
5. No exception handling is implemented.
6. Price and quantity values are not validated.

Impact:

These issues can lead to system crashes, duplicate records, and inconsistent inventory data.

Suggested Fix (Conceptual Java Approach):

```
@Transactional
public Product createProduct(ProductRequest request) {
    if (request.getName() == null || request.getSku() == null) {
        throw new RuntimeException("Missing required fields");
    }

    if (productRepository.existsBySku(request.getSku())) {
        throw new RuntimeException("SKU already exists");
    }

    Product product = new Product();
    product.setName(request.getName());
    product.setSku(request.getSku());
    product.setPrice(request.getPrice());

    productRepository.save(product);

    Inventory inventory = new Inventory();
    inventory.setProductId(product.getId());
    inventory.setWarehouseId(request.getWarehouseId());
    inventory.setQuantity(request.getInitialQuantity());

    inventoryRepository.save(inventory);

    return product;
}
```

Part 2: Database Design

The database is designed to support multiple warehouses, products, suppliers, and inventory tracking.

Tables:

Companies: id, name

Warehouses: id, company_id, name, location

Products: id, name, sku (unique), price

Inventory: product_id, warehouse_id, quantity

Suppliers: id, name, contact_email

Product_Supplier: product_id, supplier_id

Inventory_Logs: product_id, warehouse_id, change, timestamp

Explanation:

The Inventory table is separated to support multiple warehouses. SKU is unique to prevent duplication. Inventory logs are used to track stock changes.

Questions:

1. Can a product have multiple suppliers?
2. How is low stock threshold defined?
3. What is considered recent sales?
4. Should inventory be reserved for pending orders?

Part 3: Low Stock Alerts API (Spring Boot)

This API returns products that are low in stock and have recent sales activity.

```
@GetMapping("/{companyId}/alerts/low-stock")
public Map<String, Object> getAlerts(@PathVariable Long companyId) {

    List<Map<String, Object>> alerts = new ArrayList<>();

    List<Warehouse> warehouses = warehouseRepository.findByCompanyId(companyId);

    for (int i = 0; i < warehouses.size(); i++) {

        Warehouse wh = warehouses.get(i);
        List<Inventory> inventories = inventoryRepository.findByWarehouseId(wh.getId());

        for (int j = 0; j < inventories.size(); j++) {

            Inventory inv = inventories.get(j);
            Product product = productRepository.findById(inv.getProductId()).orElse(null);

            if (product == null) continue;

            int threshold = product.getThreshold() != null ? product.getThreshold() : 10;

            if (inv.getQuantity() >= threshold) continue;

            int recentSales = salesRepository.countRecentSales(
                product.getId(),
                LocalDateTime.now().minusDays(30)
            );

            if (recentSales == 0) continue;

            Supplier supplier = supplierRepository.findById(product.getId());
```

```

        int daysUntilStockout = inv.getQuantity() / Math.max(recentSales, 1);

        Map<String, Object> alert = new HashMap<>();
        alert.put("product_id", product.getId());
        alert.put("product_name", product.getName());
        alert.put("sku", product.getSku());
        alert.put("warehouse_id", wh.getId());
        alert.put("warehouse_name", wh.getName());
        alert.put("current_stock", inv.getQuantity());
        alert.put("threshold", threshold);
        alert.put("days_until_stockout", daysUntilStockout);

        Map<String, Object> supplierData = new HashMap<>();
        if (supplier != null) {
            supplierData.put("id", supplier.getId());
            supplierData.put("name", supplier.getName());
            supplierData.put("contact_email", supplier.getEmail());
        }

        alert.put("supplier", supplierData);
        alerts.add(alert);
    }

    Map<String, Object> response = new HashMap<>();
    response.put("alerts", alerts);
    response.put("total_alerts", alerts.size());

    return response;
}

```

Assumptions:

Recent sales are considered within the last 30 days. Each product has a threshold value.

Edge Cases:

1. No warehouses found
2. No inventory records
3. No recent sales
4. Supplier information missing